FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO

Facultad de Ingeniería Mecánica y

Docente:

Daniela Natali García Carreño

Materia:

Programación Web

Manual Técnico

Matricula	Alumno	Carrera
2226704	Hernández Hernández José Uriel	ITS
2097304	Esparza Leyva Jaime Sebastián	ITS
2169087	Chávez Arredondo Carlos Miguel	ITS
2226880	García Avalos Cristian Alelí	ITS

Semestre:

Agosto – diciembre 2026

Grupo: 004

Cd. Universidad, San Nicolás de los Garza, N.L.

Contenido

Introducción 4
Objetivo del proyecto 4
Herramientas utilizadas para el desarrollo del sistema 4
Punto para cada herramienta 5
Instalación 9
Ejecución e importación del proyecto. 11
Comportación de Base de Datos. 14
Preguntas Frecuentes (FAQ) 14
Conclusión 15
Anexos. 15

Introducción

Este proyecto consiste en el desarrollo de un sistema web orientado a la gestión integral de una agenda médica. La plataforma permite la administración de pacientes, consultas, recetas médicas e historial clínico, guardando todos los datos en una base de datos en la nube para garantizar su alta disponibilidad y protección.

El sistema está diseñado para facilitar el control operativo tanto para el personal médico como para los pacientes. Para lograr esto, se incorpora un sistema de control de acceso basado en roles, asegurando que cada tipo de usuario interactúe únicamente con las funciones y permisos que le corresponden.

Este documento sirve como puente entre el desarrollo del software y el Manual de Usuario, garantizando que cualquier desarrollador, administrador de sistemas o equipo de soporte posea la información técnica necesaria para compilar, configurar y dar mantenimiento al código fuente.

Objetivo del proyecto

Desarrollar e implementar una aplicación web eficiente e intuitiva que centralice y automatice la gestión administrativa y operativa de una clínica médica. El sistema busca optimizar el registro de pacientes, la programación de consultas, la generación de recetas y el control del historial clínico, garantizando la persistencia e integridad de los datos en tiempo real para mejorar la atención médica.

Herramientas utilizadas para el desarrollo del sistema

Para asegurar la correcta compilación y ejecución del proyecto, el equipo de desarrollo o administrador debe contar con los siguientes requerimientos mínimos y herramientas de software.

Requisitos mínimos del equipo (Hardware y SO):

- **Sistema Operativo:** Windows 10/11, macOS (Catalina o superior) o distribuciones de Linux basadas en Debian/Ubuntu.
- **Procesador:** Intel Core i3 / AMD Ryzen 3 o superior (arquitectura de 64 bits).
- **Memoria RAM:** Mínimo 4 GB (Se recomiendan 8 GB para mantener el entorno de desarrollo y la base de datos fluidos).
- **Almacenamiento:** Al menos 2 GB de espacio libre en disco para el entorno, repositorios y dependencias.

- **Conexión a internet:** Requerida para la validación de dependencias y la conexión con la base de datos en la nube.
- **Navegador web:** Cualquier navegador con la última versión.

Tecnología de programación

- **Frontend (Interfaz de Usuario):** HTML, CSS y JavaScript.
- **Backend / Base de Datos:** Supabase (utilizando PostgreSQL para la gestión relacional de pacientes, citas e historiales).
- **Entorno de Desarrollo Integrado (IDE):** Visual Studio Code.
- **Navegador web para pruebas:** Google Chrome o Mozilla Firefox (últimas versiones con soporte para herramientas de desarrollador).
- **Compartir el código:** Para tener el código fuente trabajado, se ocupa la plataforma Github

Inteligencia Artificial Utilizada

Durante el ciclo de desarrollo de este sistema, se hizo uso de herramientas de Inteligencia Artificial como apoyo para la optimización de código, estructuración de consultas a la base de datos y resolución de errores.

- **Herramienta:** Gemini
- **Versión:** Gemini 1.5 Pro
- **Licencia / Suscripción:** Se puede usar la versión gratuita, pero, se usó la inteligencia artificial con suscripción pro

Punto para cada herramienta

HTML, CSS y JavaScript: Lenguajes estándar de la web utilizados para construir la estructura, el diseño visual y la lógica interactiva.

- **Descarga**: No requieren descarga externa, ya que son interpretados nativamente por cualquier navegador web.
- **Elementos necesarios**: Un editor de código (como VS Code) y un navegador web para su ejecución.

Supabase (Backend y Base de Datos): Plataforma basada en PostgreSQL que gestiona la base de datos relacional y la autenticación.

- **Descarga:** No requiere instalación local de software, ya que es un servicio en la nube (BaaS).

- **Elementos necesarios:** Crear una cuenta en su sitio web oficial (https://supabase.com/) y generar un proyecto para obtener la URL y la API Key pública necesarias para la conexión.

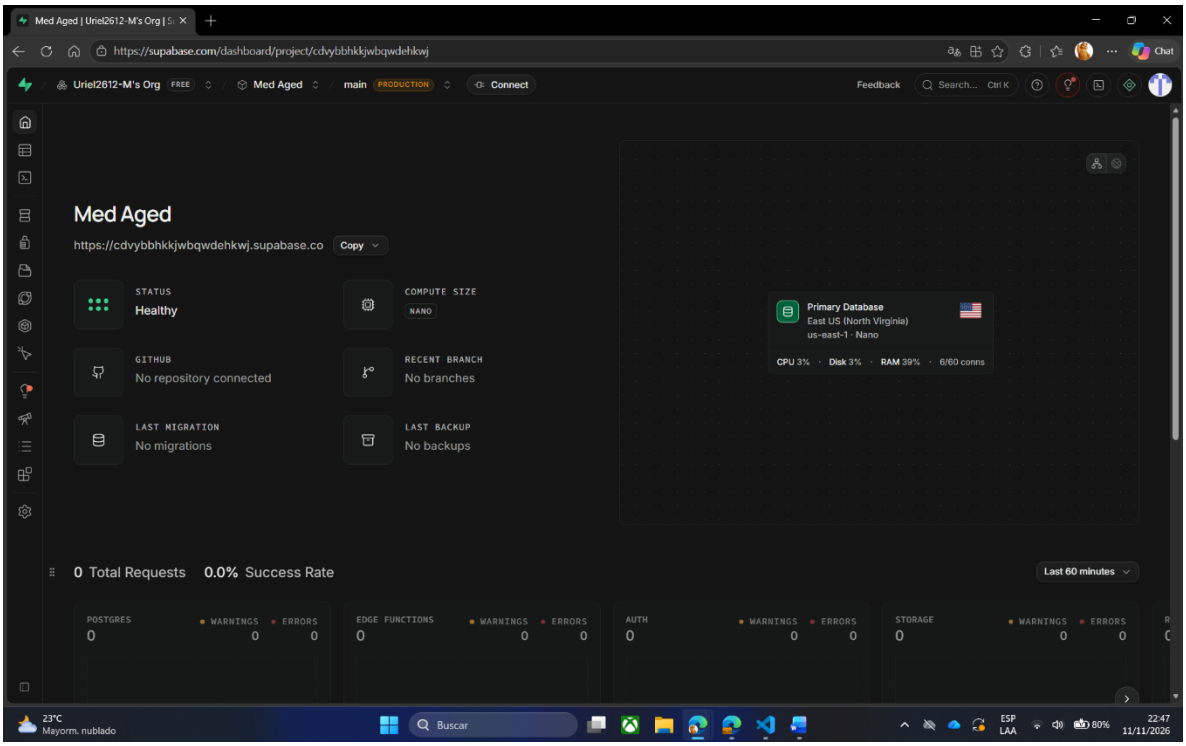


Ilustración 1. Creación de cuenta en supabase

Visual Studio Code (Entorno de Desarrollo): Editor de código fuente ligero utilizado para escribir, editar y depurar el proyecto.

- **Descarga:** Se descarga de forma gratuita desde su página oficial (https://code.visualstudio.com/).

- **Elementos necesarios:** Para este proyecto web, es estrictamente recomendado instalar la extensión **"Live Server"** dentro del editor, la cual permite montar un servidor local rápido para visualizar los cambios de HTML/JS en tiempo real sin recargar la página manualmente.

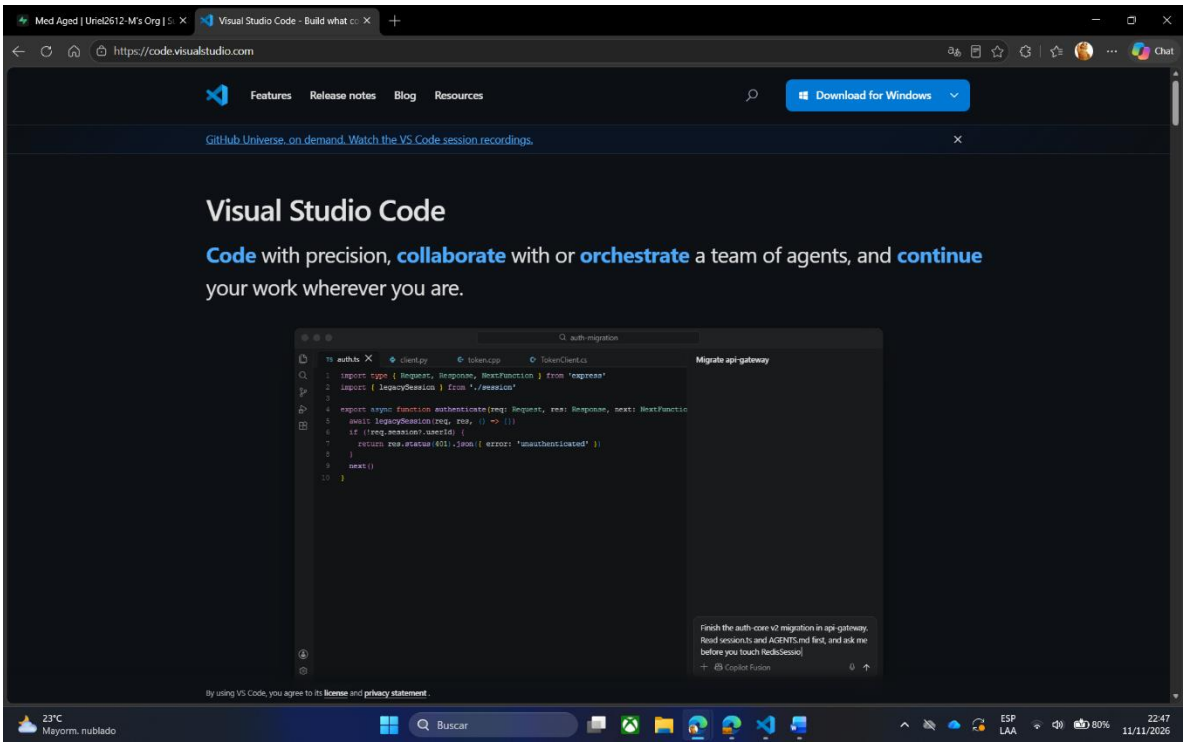


Ilustración 2. Página oficial para descargar Visual Studio Code

Extensión utilizada para Visual Studio Code fue la siguiente:

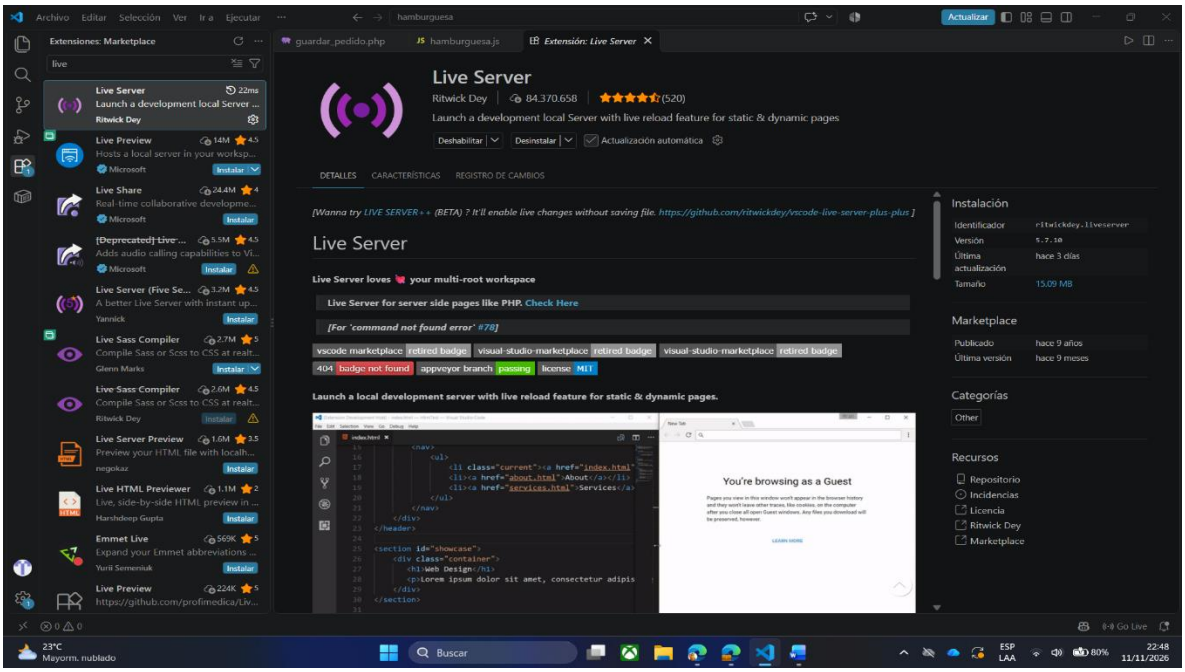


Ilustración 3. Extensión para poder ver la página en el navegador

Navegadores Web (Google Chrome / Microsoft edge): Utilizados para ejecutar la aplicación y verificar su funcionamiento.

- **Descarga:** Desde sus sitios oficiales (google.com/chrome), en el caso de Microsoft no es necesaria alguna descarga.
- **Elementos necesarios:** Tener habilitadas y conocer el uso de las Herramientas de Desarrollador (DevTools) incluidas por defecto (presionando F12), fundamentales para depurar el código JavaScript y ajustar el CSS.

GitHub y Git: Plataforma en la nube y sistema de control de versiones para el trabajo colaborativo sobre el código fuente.

- **Descarga:** Se requiere descargar **Git** para la línea de comandos (https://git-scm.com/downloads) o la interfaz gráfica **GitHub Desktop** (https://desktop.github.com/).
- **Elementos necesarios:** Una cuenta activa en GitHub (https://github.com/) y credenciales de acceso (Tokens) configuradas en el equipo local para poder clonar (git clone) y subir cambios al repositorio.

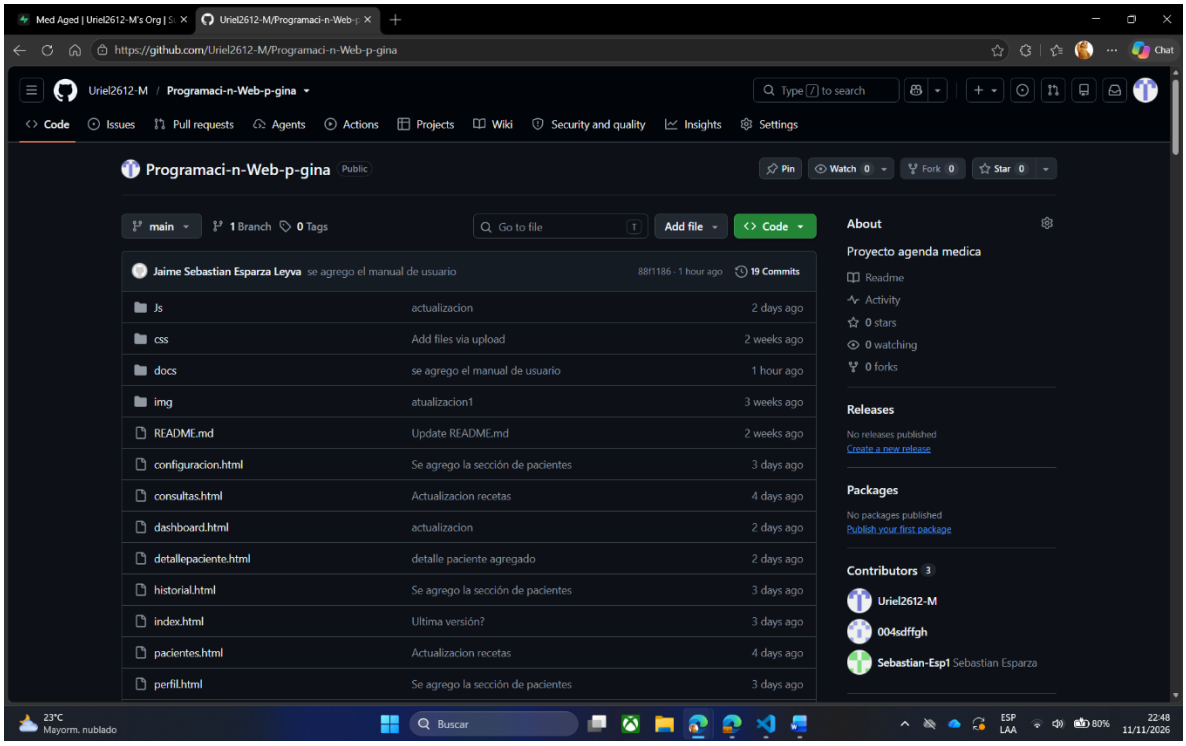


Ilustración 4. Cuenta en Github para la creación de la aplicación

Gemini 1.5 Pro (IA de apoyo): Asistente utilizado para la optimización de código y estructuración de consultas SQL.

- **Descarga:** Se accede vía web (https://gemini.google.com/), no requiere descarga.
- **Elementos necesarios:** Cuenta de Google y suscripción activa a la versión Advanced/Pro.

Instalación

La instalación y configuración del entorno se divide en dos partes: la preparación del código en el equipo local y la configuración del backend en la nube. Siga estos pasos en orden para preparar el sistema:

Paso 1: Obtención del código fuente (Entorno Local)

1. Abra la terminal de su sistema operativo (Símbolo del sistema, PowerShell o Git Bash).
2. Navegue mediante comandos hasta el directorio donde desea guardar el proyecto (por ejemplo: cd Documentos/Proyectos).
3. Ejecute el siguiente comando para descargar el repositorio: git clone <https://github.com/Uriel2612-M/Programaci-n-Web-p-gina/settings>
4. Ingrese a la carpeta del proyecto recién descargado: cd <Programaci-n-Web-p-gina>

Paso 2: Configuración del Entorno de Desarrollo (Visual Studio Code)

1. Abra el editor **Visual Studio Code**.
2. Diríjase al menú Archivo > Abrir carpeta... (o File > Open Folder...) y seleccione la carpeta del proyecto que clonó en el paso anterior.
3. Vaya al panel de Extensiones (Ctrl + Shift + X) y asegúrese de tener instalada y habilitada la extensión **Live Server**.

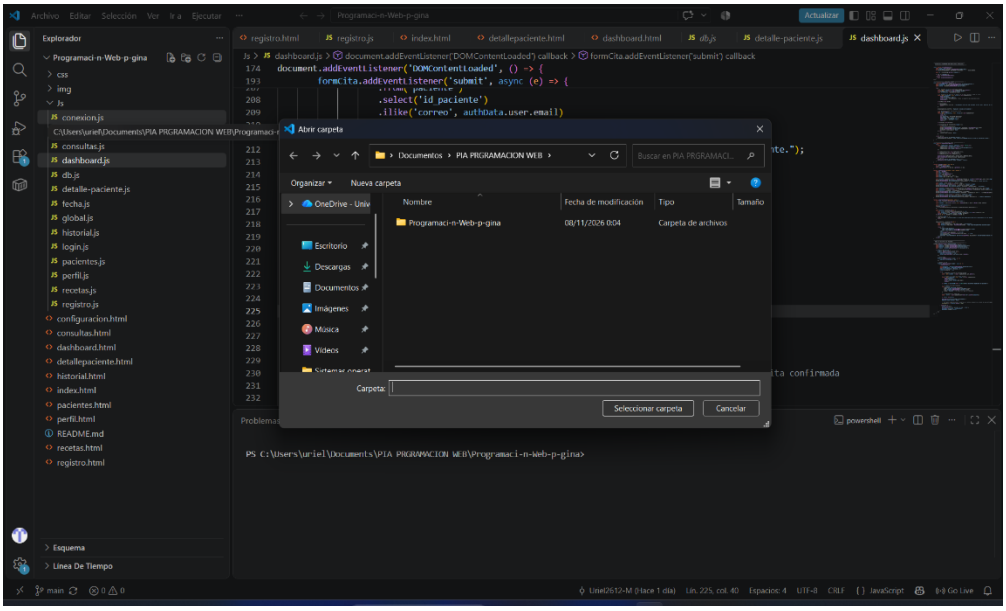


Ilustración 5. Clonar el repositorio en nuestro entorno local.

Paso 3: Configuración de la Base de Datos (En la nube - Supabase)

1. Ingrese a su cuenta en https://supabase.com/ y haga clic en **New Project**.
2. Asigne un nombre al proyecto, genere una contraseña segura para la base de datos y seleccione la región del servidor.
3. Una vez creado el proyecto, diríjase a la sección **SQL Editor** en el panel izquierdo. Copie y ejecute el script SQL (ubicado en los Anexos de este manual) para generar automáticamente todas las tablas relacionales (Paciente, Cita, Médico, etc.).
4. Diríjase a Project Settings > API. Copie los valores de **Project URL** y **Project API Key (anon, public)**.

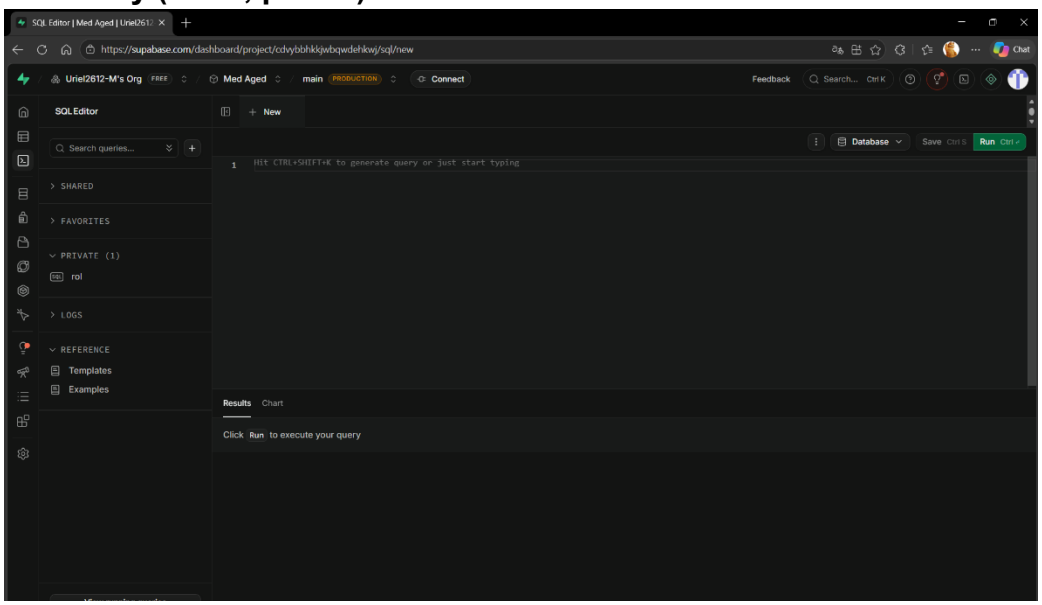


Ilustración 6. Insertar código SQL para supabase

Paso 4: Conexión del frontend con la base de datos

1.

2. Pegue las credenciales obtenidas en el paso anterior dentro de las constantes de configuración correspondientes:

const supabaseUrl = ‘https://cdvybbhkkjwbqwdehkwj.supabase.co'; const

supabaseAnonKey=‘sb_publishable_EF18xTN_tGknNqnURmElUw_J5uBmFXr’ ;

3. Guarde los cambios (Ctrl + S). El sistema ya está instalado y enlazado a la base de datos.

Ejecución e importación del proyecto.

Una vez realizada la instalación siguiendo los pasos anteriormente mencionados. El proyecto esta listo para ejecutarse en un entorno local.

Al ser un proyecto realizado con HTML, CSS y JavaScript no es necesario la compilación de un servidor backend complejo, sino que, al quererlo correr en un entorno de pruebas, con la extensión **Live server** de Visual Studio Code para montarlo en el servidor local.

Para poder realizar esto, es necesario seguir los siguientes pasos;

1. Dentro de Visual Studio Code (con la carpeta del proyecto abierta), localice el archivo principal de la interfaz en el panel explorador del lado izquierdo (en este caso llamado index.html)

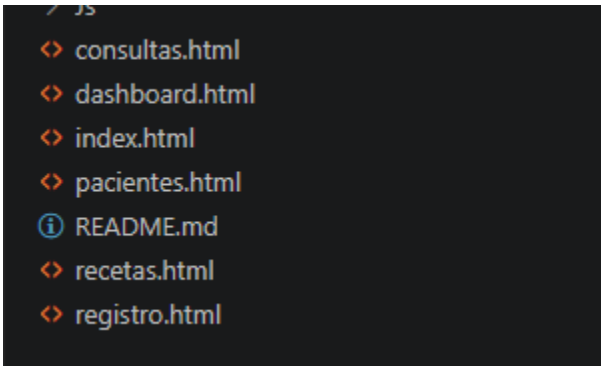


Ilustración 7. Archivo principal de la página

2. Haga clic derecho sobre dicho archivo.

3. En el menú desplegable, seleccione la opción **"Open with Live Server"** (o presione la combinación de teclas Alt + L, Alt + O).
4. Automáticamente se abrirá su navegador web predeterminado (Microsoft edge) en la dirección local del puerto 5500 (por ejemplo: http://127.0.0.1:5500/index.html), mostrando la interfaz del sistema conectada y operativa.

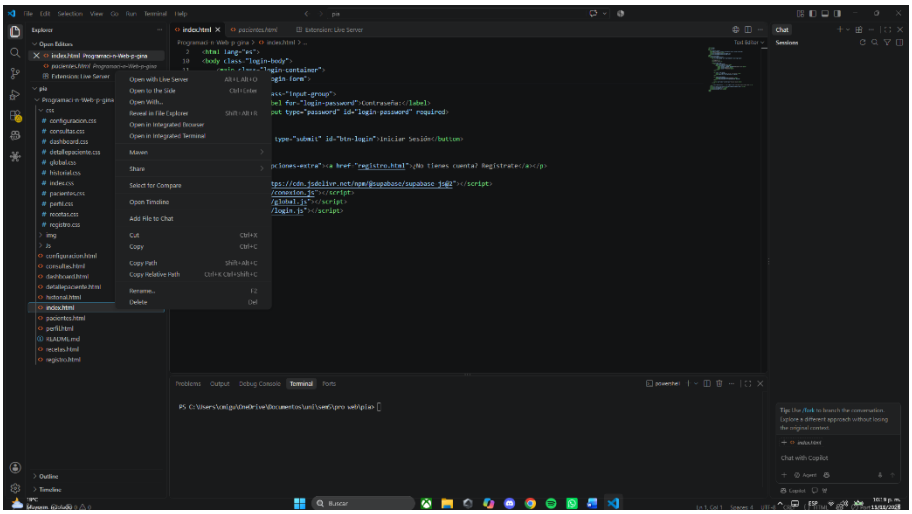


Ilustración 8. Habilitar extensión "Live Server"

A continuación, se presentan las vistas principales del sistema en ejecución, demostrando la correcta importación y funcionamiento del código:

- **Pantalla de Inicio** / **Acceso:***(En esta vista el usuario ingresa sus credenciales para acceder al sistema)*

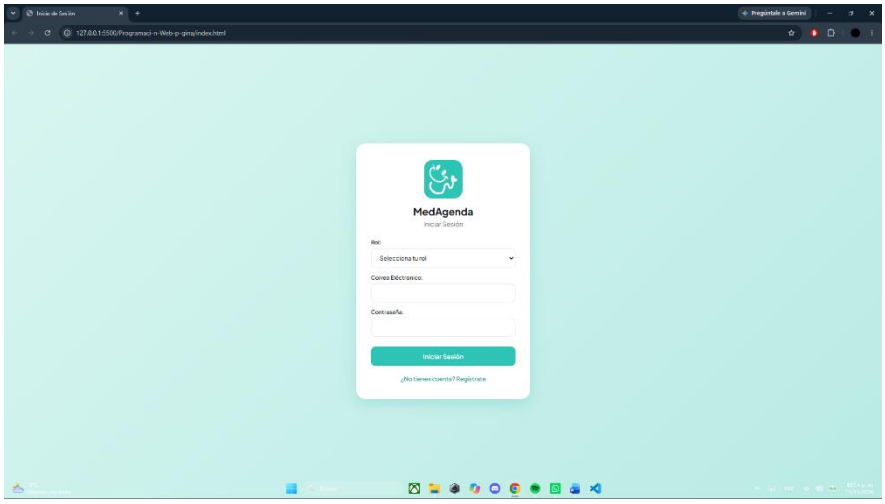


Ilustración 9. Página principal.

- **Panel de Control (Dashboard):***(Código principal del dashboard para visualizar su funcionamiento).*

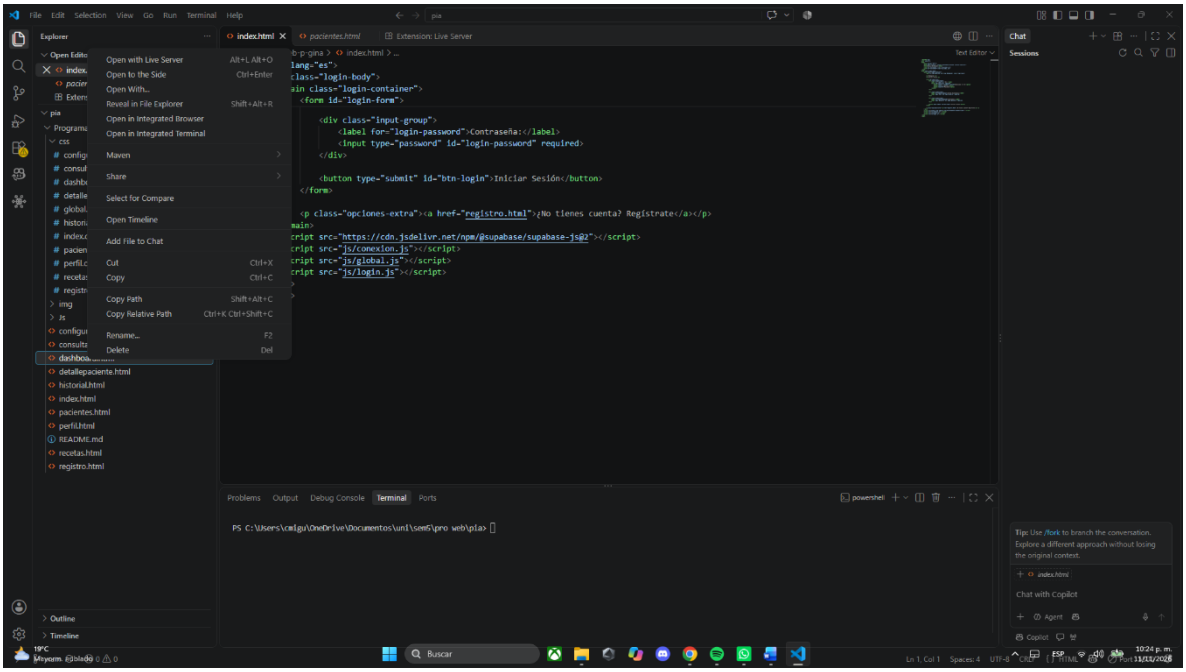


Ilustración 10. Código Dashboard inicio de la aplicación

- **Registro / Gestión de Datos:***(Ejemplo de un formulario en funcionamiento, como el de agregar un paciente o agendar una cita médica. Los datos ingresados aquí se envían directamente a Supabase)*

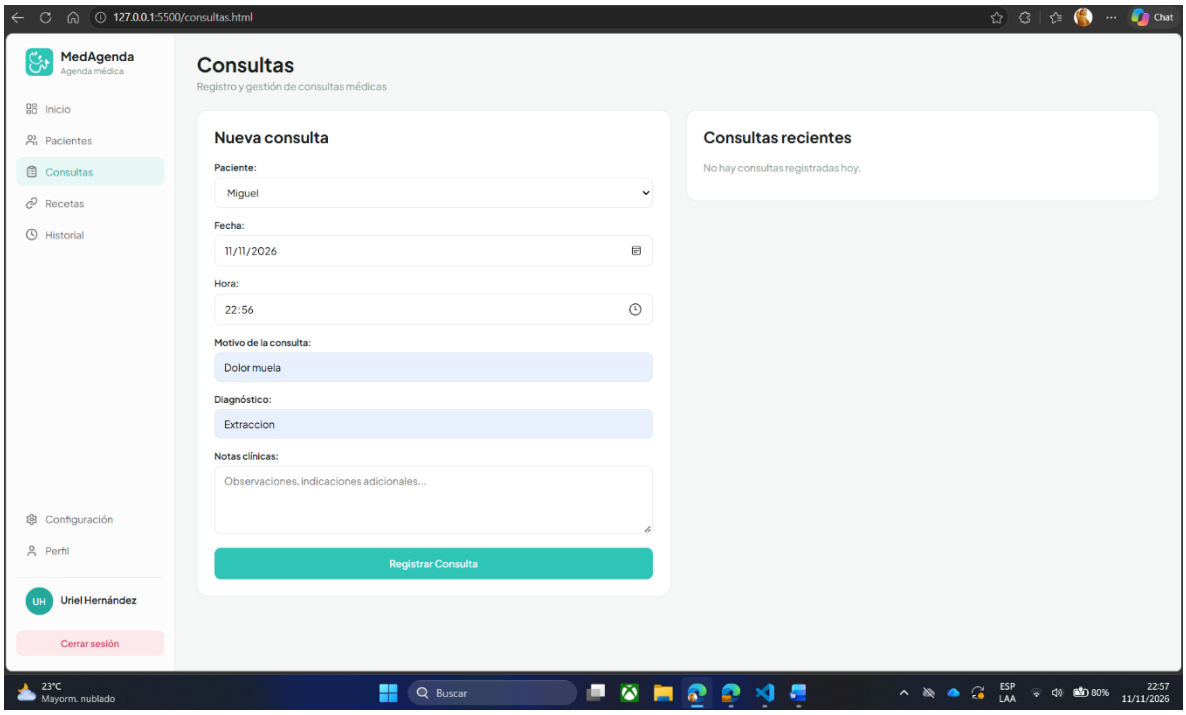


Ilustración 11. Formulario funcional de nuestra aplicación

Comportación de Base de Datos.

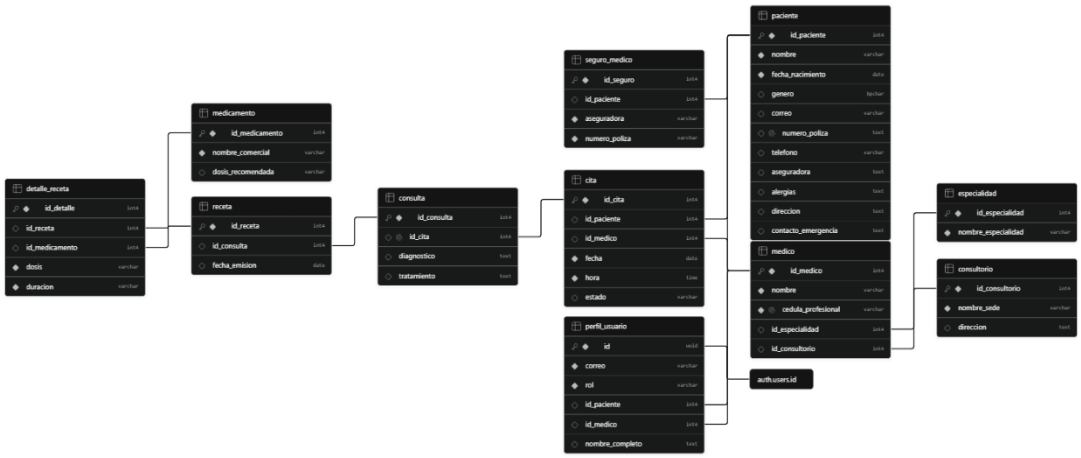


Ilustración 12. Diagrama entidad relación de Supabase

Preguntas Frecuentes (FAQ)

¿Es necesario instalar algún servidor local como XAMPP o WAMP para ejecutar el proyecto? No. Al estar desarrollado con tecnologías web estándar del lado del cliente (HTML, CSS y JavaScript) y utilizar Supabase como servicio de backend en la nube, basta con abrir el proyecto en Visual Studio Code y ejecutarlo mediante la extensión *Live Server*.

¿Por qué la página no guarda o no muestra los registros al iniciar sesión o llenar un formulario? Esto ocurre generalmente si no hay conexión a internet o si las credenciales de conexión (URL del proyecto y llave pública anon key de Supabase) fueron modificadas o eliminadas en el archivo conexion.js dentro de la carpeta js. Verifica que tengas acceso a la red y que el proyecto en Supabase se encuentre activo.

¿Qué hago si al intentar entrar directamente a dashboard.html o pacientes.html el sistema me regresa a la pantalla de inicio? El sistema cuenta con validación de sesión para proteger la información médica. Para visualizar las vistas internas correctamente, debes iniciar sesión desde index.html con un usuario válido o registrar una cuenta nueva desde la opción de registro.

¿Dónde se almacenan los datos de los pacientes, médicos, citas y recetas? Toda la información se almacena de forma estructurada y persistente en una base de datos relacional PostgreSQL alojada en **Supabase**, permitiendo realizar consultas, altas y actualizaciones en tiempo real desde la interfaz web.

¿Cómo puedo actualizar el código si mis compañeros realizan cambios en el repositorio? Abre la terminal dentro de la carpeta del proyecto en Visual Studio Code y ejecuta el comando git pull origin main (o el nombre de la rama principal) para descargar e integrar las últimas modificaciones sin perder la estructura local.

Conclusión

El desarrollo e implementación del sistema web **MedAgenda** permitió integrar de manera práctica los fundamentos del desarrollo frontend (HTML, CSS y JavaScript) con una arquitectura backend moderna en la nube mediante Supabase. A lo largo de este proyecto se logró construir una plataforma funcional orientada a la gestión médica, capaz de administrar el control de acceso por roles, el registro de pacientes, la programación de citas y la emisión de consultas y recetas bajo un modelo de base de datos relacional íntegro y seguro.

Asimismo, la elaboración de este manual técnico documenta de forma clara la estructura del código fuente, el despliegue en entornos locales de prueba mediante herramientas como *Live Server* y el flujo operativo de la aplicación, facilitando tanto su mantenimiento futuro como su comprensión para cualquier usuario o desarrollador que desee replicar o escalar el sistema.

Anexos.

Anexo A. Diccionario de Datos de las Tablas Principales

La base de datos relacional implementada en Supabase (PostgreSQL) se encuentra estructurada mediante las siguientes tablas y campos principales para la gestión del consultorio médico:

- **Tabla PACIENTE**: Almacena la información personal y clínica de cada paciente.
    - id_paciente (PK): Identificador único autoincrementable.

    - nombre: Nombre completo del paciente.
    - fecha_nacimiento: Fecha de nacimiento para el cálculo de edad.
    - curp: Clave Única de Registro de Población.
    - telefono: Número de contacto telefónico.
    - tipo_sangre: Grupo sanguíneo y factor RH del paciente.
    - alergias: Antecedentes de reacciones alérgicas relevantes.
- **Tabla MEDICO**: Contiene los datos del personal de salud registrado en el sistema.
    - id_medico (PK): Identificador único del médico.
    - nombre: Nombre completo del profesional.
    - cedula_profesional: Número de cédula legal acreditada.
    - id_especialidad (FK): Relación con el área médica correspondiente.
    - id_consultorio (FK): Relación con la sede o consultorio asignado.
- **Tabla CITA**: Gestiona la agenda y programación de citas médicas.
    - id_cita (PK): Identificador único de la cita.
    - id_paciente (FK): Relación con el paciente citado.
    - id_medico (FK): Relación con el médico asignado.
    - fecha y hora: Momento programado para la atención.
    - estado: Estatus actual de la cita (Pendiente, Completada, Cancelada).

Anexo B. Especificaciones Técnicas y Dependencias del Entorno

Para la correcta ejecución, compilación y despliegue del sistema web, se emplearon las siguientes herramientas y librerías externas:

- **Lado del Cliente (Frontend):**
    - Estructura de marcado en **HTML5** y hojas de estilo en **CSS3** orientadas al diseño responsivo.
    - Interactividad y lógica de negocio mediante **JavaScript** nativo (ES6+).
- **Conexión a Datos (Backend as a Service):**
    - Integración con **Supabase** mediante su SDK oficial por CDN (@supabase/supabase-js) para la autenticación y operaciones CRUD en tiempo real.

- **Entorno de Desarrollo Local:**
    - Editor de código fuente **Visual Studio Code**.
    - Servidor de desarrollo local mediante la extensión **Live Server** (puerto 5500).
    - Control de versiones local y remoto mediante **Git y GitHub**.

Anexo C. Enlaces del Proyecto

- **Repositorio del Código Fuente (GitHub):** https://github.com/usuario/MedAgenda
- **Plataforma de Base de Datos en la Nube:** Consola de administración y gestión en **Supabase** (supabase.com).

Anexo D. Código SQL.

-- WARNING: This schema is for context only and is not meant to be run.

-- Table order and constraints may not be valid for execution.

CREATE TABLE public.especialidad (

id_especialidad integer NOT NULL DEFAULT nextval('especialidad_id_especialidad_seq'::regclass),

nombre_especialidad character varying NOT NULL,

CONSTRAINT especialidad_pkey PRIMARY KEY (id_especialidad)

);

CREATE TABLE public.consultorio (

id_consultorio integer NOT NULL DEFAULT nextval('consultorio_id_consultorio_seq'::regclass),

nombre_sede character varying NOT NULL,

direccion text,

CONSTRAINT consultorio_pkey PRIMARY KEY (id_consultorio)

);

CREATE TABLE public.medicamento (

id_medicamento integer NOT NULL DEFAULT nextval('medicamento_id_medicamento_seq'::regclass),

```
  nombre_comercial character varying NOT NULL,
  dosis_recomendada character varying,
  CONSTRAINT medicamento_pkey PRIMARY KEY (id_medicamento)
);
CREATE TABLE public.paciente (
  id_paciente integer NOT NULL DEFAULT nextval('paciente_id_paciente_seq'::regclass),
  nombre character varying NOT NULL,
  fecha_nacimiento date NOT NULL,
  genero character,
  correo character varying,
  numero_poliza text UNIQUE,
  telefono character varying,
  aseguradora text,
  alergias text,
  direccion text,
  contacto_emergencia text,
  CONSTRAINT paciente_pkey PRIMARY KEY (id_paciente)
);
CREATE TABLE public.medico (
  id_medico integer NOT NULL DEFAULT nextval('medico_id_medico_seq'::regclass),
  nombre character varying NOT NULL,
  cedula_profesional character varying NOT NULL UNIQUE,
  id_especialidad integer,
  id_consultorio integer,
  CONSTRAINT medico_pkey PRIMARY KEY (id_medico),
  CONSTRAINT medico_id_especialidad_fkey FOREIGN KEY (id_especialidad) REFERENCES public.especialidad(id_especialidad),
```

```
  CONSTRAINT medico_id_consultorio_fkey FOREIGN KEY (id_consultorio) REFERENCES public.consultorio(id_consultorio)

);

CREATE TABLE public.seguro_medico (

  id_seguro integer NOT NULL DEFAULT nextval('seguro_medico_id_seguro_seq'::regclass),

  id_paciente integer,

  aseguradora character varying NOT NULL,

  numero_poliza character varying NOT NULL,

  CONSTRAINT seguro_medico_pkey PRIMARY KEY (id_seguro),

  CONSTRAINT seguro_medico_id_paciente_fkey FOREIGN KEY (id_paciente) REFERENCES public.paciente(id_paciente)

);

CREATE TABLE public.cita (

  id_cita integer NOT NULL DEFAULT nextval('cita_id_cita_seq'::regclass),

  id_paciente integer,

  id_medico integer,

  fecha date NOT NULL,

  hora time without time zone NOT NULL,

  estado character varying DEFAULT 'Pendiente'::character varying,

  CONSTRAINT cita_pkey PRIMARY KEY (id_cita),

  CONSTRAINT cita_id_paciente_fkey FOREIGN KEY (id_paciente) REFERENCES public.paciente(id_paciente),

  CONSTRAINT cita_id_medico_fkey FOREIGN KEY (id_medico) REFERENCES public.medico(id_medico)

);

CREATE TABLE public.consulta (

  id_consulta integer NOT NULL DEFAULT nextval('consulta_id_consulta_seq'::regclass),

  id_cita integer UNIQUE,
```

```
  diagnostico text,
  tratamiento text,
  CONSTRAINT consulta_pkey PRIMARY KEY (id_consulta),
  CONSTRAINT consulta_id_cita_fkey FOREIGN KEY (id_cita) REFERENCES public.cita(id_cita)
);
CREATE TABLE public.receta (
  id_receta integer NOT NULL DEFAULT nextval('receta_id_receta_seq'::regclass),
  id_consulta integer,
  fecha_emision date DEFAULT CURRENT_DATE,
  CONSTRAINT receta_pkey PRIMARY KEY (id_receta),
  CONSTRAINT receta_id_consulta_fkey FOREIGN KEY (id_consulta) REFERENCES public.consulta(id_consulta)
);
CREATE TABLE public.detalle_receta (
  id_detalle integer NOT NULL DEFAULT nextval('detalle_receta_id_detalle_seq'::regclass),
  id_receta integer,
  id_medicamento integer,
  dosis character varying NOT NULL,
  duracion character varying NOT NULL,
  CONSTRAINT detalle_receta_pkey PRIMARY KEY (id_detalle),
  CONSTRAINT detalle_receta_id_receta_fkey FOREIGN KEY (id_receta) REFERENCES public.receta(id_receta),
  CONSTRAINT detalle_receta_id_medicamento_fkey FOREIGN KEY (id_medicamento) REFERENCES public.medicamento(id_medicamento)
);
CREATE TABLE public.perfil_usuario (
  id uuid NOT NULL,
  correo character varying NOT NULL,
```

```
  rol character varying NOT NULL,
  id_paciente integer,
  id_medico integer,
  nombre_completo text,
  CONSTRAINT perfil_usuario_pkey PRIMARY KEY (id),
  CONSTRAINT perfil_usuario_id_fkey FOREIGN KEY (id) REFERENCES auth.users(id),
  CONSTRAINT perfil_usuario_id_paciente_fkey FOREIGN KEY (id_paciente) REFERENCES public.paciente(id_paciente),
  CONSTRAINT perfil_usuario_id_medico_fkey FOREIGN KEY (id_medico) REFERENCES public.medico(id_medico)
);
```